Instructor Guide – Introduction to Model-Driven Network Automation Using NETCONF

Lab Overview (Instructor Reference)

This lab introduces students to model-driven network automation using NETCONF and Python.

Students interact with a single Cisco IOS XE router (CSR1000v or NDG NETLAB+ device) to:

- Retrieve structured configuration data
- Apply a safe, non-disruptive configuration change
- Verify post-change device state

The lab emphasizes:

- Automation intent
- Configuration safety
- Structured data validation
- The complete automation lifecycle: retrieve → modify → verify

---

Instructor Preparation

Supported Delivery Methods

- NDG NETLAB+ DevNet Pod
- Local VirtualBox deployment (preferred for Capstone validation)

Local VirtualBox Requirements

If running locally, ensure:

- DEVASC-LABVM is operational
- CSR1000v virtual router is running
- DEVASC VM can reach the router management interface
- NETCONF is enabled (`netconf-yang`)
- SSH is operational

- TCP port 830 is reachable

Instructor Note: The DEVASC VM provides automation tools only. A separate IOS XE device is required for NETCONF-based automation.

Pre-Lab Validation (Instructor Checklist)

Before students begin **Part 1**, confirm:

- CSR1000v router boots successfully
- Management interface is up/up
- NETCONF connectivity is functional
- Python and the `ncclient` library are installed on the DEVASC VM

This preparation ensures students focus on automation concepts rather than environment troubleshooting.

Instructor Pre-Lab: CSR1000v Initial Configuration *(Local VirtualBox Delivery Only)*

This section applies only when the lab is delivered using VirtualBox with a locally hosted CSR1000v router.
No router preparation is required when using the NDG NETLAB+ DevNet pod.

Instructor Note: This preparation should be completed before students begin **Part 1** of the lab. Students do not perform these steps during the automation exercises unless explicitly instructed.

Instructor Preparation Overview

Before any NETCONF or Python automation tasks, the CSR1000v router must be initialized and accessible via the console. This ensures the device is operational and ready for management-plane configuration.

Step 1: Access Router Console and Enter Privileged EXEC Mode

Instructor Expectations

- The CSR1000v virtual machine powers on successfully
- IOS XE completes the boot process without errors

- The router presents a `CSR1kv>` prompt

Student Action (Observed)

- Students access the VirtualBox console
- Students enter privileged EXEC mode using the `enable` command

Successful Completion Criteria

- Router prompt changes from `CSR1kv>` to `CSR1kv#`
- No configuration changes are made at this stage

Note: Informational messages related to cryptographic services, licensing, or Plug-and-Play (PnP) discovery are normal and should be ignored.

Step 2: Enter Global Configuration Mode and Set Hostname

Instructor Expectations

- Students enter global configuration mode
- The router hostname is updated to match the lab environment

Student Action (Observed)

- Students enter configuration mode using `configure terminal`
- Students set the hostname to `CSR1000v`

Successful Completion Criteria

- Router prompt reflects configuration mode
- Hostname updates to `CSR1000v(config)#`

Note: SMART licensing messages that appear after changing the hostname are informational and do not require corrective action.

Instructor Verification Checkpoint

Before students proceed to NETCONF or Python automation, confirm:

- CSR1000v router is operational
- Console access is functional
- Hostname configuration is complete

Students should not proceed until this checkpoint is met.

Lab Objectives (Instructor Reference)

By the end of this lab, students should be able to:

- Compare CLI-based automation and model-driven automation
- Retrieve structured configuration data using NETCONF
- Apply a safe interface-level configuration change
- Verify device state after automation execution
- Explain how structured automation improves reliability

Estimated Completion Time

45–75 minutes

Part 1: Verify the Automation Environment

Student Action

- Launch the NDG DevNet pod

  OR
- Power on the DEVASC-LABVM and CSR1000v virtual machines (Local VirtualBox delivery)
- Open the DEVASC VM terminal
- Execute the NETCONF connectivity test script:

  ```
  python3 test_netconf.py
  ```

Expected Result

- The script executes successfully.
- A NETCONF session to the IOS XE device is established.
- No configuration changes are applied to the device during this step.

Instructor Notes:

This step validates that the Python environment, ncclient library, and NETCONF connectivity are all functional before students begin automation tasks.
The script establishes a NETCONF session but performs no `<edit-config>` operations and does not modify device configuration. Successful execution confirms readiness for **Parts 2–4**.

Common Issues

- Missing `ncclient` module in the DEVASC VM
- Incorrect device IP address or credentials in the script
- NETCONF (`netconf-yang`) not enabled on the IOS XE device
- Students using the wrong Python interpreter (e.g., `python` instead of `python3`)

Part 2: Retrieve Interface Configuration Using NETCONF

Student Action

- Create and run the NETCONF retrieval script (`netconf_get_interfaces.py`).
- Execute a NETCONF `<get>` operation to retrieve interface configuration data.
- Observe the structured XML output returned by the device.

Expected Result

- XML-formatted configuration data is displayed.
- Interface configuration elements (such as interface name, description, and administrative state) are visible in the structured output.
- No configuration changes are applied to the device during this step.

Instructor Note: Students may initially expect formatted CLI-style output. Reinforce that NETCONF returns structured, machine-readable XML data, which allows precise validation of device state. Emphasize that this read-only retrieval script will be reused later in **Part 4** to verify post-change configuration state.

Part 3: Apply a Safe Interface Configuration Change

Student Action

- Modify the existing NETCONF script used in **Part 2** (`netconf_get_interfaces.py`) so that it performs a NETCONF `<edit-config>` operation.
- Apply one safe, non-disruptive configuration change by updating the interface description on GigabitEthernet1.
- Execute the modified script to apply the configuration change.

Expected Result

- NETCONF returns a successful RPC reply (`<ok/>`).
- No error messages or partial failures occur.
- The device remains reachable after the configuration change.

Instructor Notes:

This part introduces **write operations** using NETCONF. Emphasize that students are intentionally replacing the read-only `<get>` operation from **Part 2** with a write-based <edit-config> operation. Reinforce that only a single, controlled change (interface description) should be made to preserve automation safety and avoid service disruption.

Common Student Errors

- Attempting to modify multiple configuration elements in a single `<edit-config>` operation.
- Forgetting to replace the `<get>` operation with `<edit-config>`.
- Selecting an incorrect interface name.
- Expecting structured XML output instead of an `<ok/>` response.

Part 4: Verify Post-Change Device State

Student Action

1. Restore and re-run the read-only NETCONF retrieval script (`netconf_get_interfaces.py`) used in **Part 2**.
2. Execute a NETCONF `<get>` operation to retrieve the current interface configuration.
3. Compare the post-change output to the pre-change state observed in **Part 2**.

Expected Result

- Structured XML output is returned by the device.
- The intended configuration change (updated interface description) is visible.
- No unintended configuration changes appear.

Instructor Note:

Verification is the most important conceptual takeaway of this lab. Students must understand that automation is incomplete without post-change validation.
Emphasize that **Part 4** must be performed using a read-only `<get>` operation, not the `<edit-config>` script from Part 3. If the `<edit-config>` logic is still present, the device will return an `<ok/>` response instead of configuration data, preventing proper verification.

Validation Checklist (Instructor Use)

- NETCONF connectivity established
- Structured configuration data retrieved using `<get>`
- Safe configuration change applied using `<edit-config>`
- Post-change verification completed using `<get>`

Instructor Note: If all conditions are met, the lab is considered successful.

Answers to Review Questions (Instructor Reference)

1.) Why is NETCONF output easier to validate than CLI output?

NETCONF returns structured, predictable data in a machine-readable format (XML), which allows specific configuration values to be easily identified and verified. In contrast, CLI output is unstructured text designed for human interpretation, making reliable validation more difficult and error-prone for automation systems.

2.) What risks exist when configuration changes are only partially applied?

Partial configuration changes may leave a device in an inconsistent or unstable state, potentially impacting network

functionality. Model-driven interfaces such as NETCONF reduce this risk by applying configuration changes atomically and providing clear success or error responses.

3.) How does structured automation improve reliability?

Structured automation improves reliability by allowing systems to clearly define intent, apply changes consistently, and verify device state before and after modifications. This reduces ambiguity, minimizes human error, and enables deterministic validation of network configurations.

Sample Reflection Response (Instructor Reference)

NETCONF improves automation reliability by enabling structured data exchange and clear validation mechanisms. Unlike CLI-based automation, which relies on unstructured text output and sequential command execution, NETCONF uses modeled data and well-defined operations to retrieve, modify, and verify device configuration. This structure allows automation systems to confirm intended state changes accurately, detect errors early, and ensure that devices remain in a consistent and predictable state after automation is applied.

Instructor Note: Script Usage and Environment Variations

All NETCONF scripts used in this lab are created, modified, and restored by students directly within the lab steps. Scripts are not distributed as external files.

When delivering the lab using NDG NETLAB+, device addressing and credentials are preconfigured and require no instructor modification.

When delivering the lab using a local VirtualBox CSR1000v or IOS XE router, instructors should ensure that students update script parameters (such as device IP address, username, and password) to match the local environment.

Instructor Guidance

- Students use a read-only <get> script in **Parts 2 and 4**.
- Students temporarily modify that script to perform a write `<edit-config>` operation in **Part 3**.
- Emphasize the importance of restoring the read-only retrieval logic before performing post-change verification.

Note: This approach reflects real-world automation workflows, where read and write operations are clearly separated by intent.